

Classroom B - C Lab Fixing Worksheet (Single File)

Student Task

Build **one single C file** named `labwork_b_fix.c`.

Requirements:

1. Use only one `#include <stdio.h>`.
2. Fix each broken example below as a separate function: `example1_data_types()` to `example10_mini_combined()`.
3. Keep helper functions when needed (for example `multiply`, `showMessage`, `rectangleArea`).
4. Write only one `main()` and call all 10 example functions inside `main()` in order.

Suggested `main()` call order:

```
int main(void) {
    example1_data_types();
    example2_variable_declaration_assignment();
    example3_declaration_with_initialization();
    example4_function_definition();
    example5_void_function();
    example6_arithmetic_operators();
    example7_relational_operators();
    example8_logical_operators();
    example9_increment_decrement_assignment();
    example10_mini_combined();
    return 0;
}
```

Example 1 - Data Types

Learning Note: Data types in C control how values are stored and interpreted. Choose a type that matches the value you want to keep.

```
void example1_data_types( {
    int year = "2026";
    float temperature = ;
    double gravity = "9.80665";
    char section = BB;
    printf("Year:%d Temp:%.1f Gravity:%.5lf Section:%c\n", year,
    temperature, gravity, section)
}
```

Hint: match variable types with literal formats and fix missing punctuation.

Example 2 - Variable Declaration and Assignment

Learning Note: First declare variables with their type, then assign values. Using a variable before declaration causes errors.

```
void example2_variable_declaration_assignment(void) {  
    p = 14;  
    int p,;  
    q = 6  
    int q;  
    printf("p=%d q=%d r=%d\n", p, q, r);  
}
```

Hint: declarations should appear before assignments.

Example 3 - Declaration with Initialization

Learning Note: Initialization gives a starting value at declaration time. Missing values in an initialized declaration break the statement.

```
void example3_declaration_with_initialization(void) {  
    int n1 = 2, n2 = , n3 = 8  
    printf("n1=%d n2=%d n3=%d\n", n1, n2, n4);  
}
```

Hint: every initialized variable needs a value.

Example 4 - Function Definition (Multiply)

Learning Note: Function parameters are typed inputs separated by commas. Correct function headers are required before calling them.

```
int multiply(int a int b) {  
    return a ** b;  
}  
  
void example4_function_definition(void) {  
    int result = multiply(5);  
    printf("Product = %d\n", result)  
}
```

Hint: verify function parameter separators in the header.

Example 5 - Void Function

Learning Note: A `void` function has no return value, but it still needs a valid function signature with parentheses.

```
void showMessage(void {  
    printf("Welcome to basic C practice!\n")  
}  
  
void example5_void_function(void) {  
    showMessage();  
}
```

Hint: function declarations need parentheses, even with no parameters.

Example 6 - Arithmetic Operators

Learning Note: Arithmetic expressions may be integers or floating-point values. Output formatting in `printf` must match the expression type.

```
void example6_arithmetic_operators(void) {  
    int x = 18, y = 5;  
    printf("x + y = %d\n", x + y);  
    printf("x - y = %d\n", x - y);  
    printf("x * y = %d\n", x * y);  
    printf("x / y = %d\n", x // y);  
    printf("x %% y = %d\n", x % y)  
}
```

Hint: `printf` format specifier must match the expression type.

Example 7 - Relational Operators

Learning Note: Operators like `>`, `<`, `==`, and `!=` compare values. Writing operators incorrectly changes meaning or causes syntax errors.

```
void example7_relational_operators(void) {  
    int a = 12, b = 12;  
    printf("a > b : %d\n", a > b);  
    printf("a < b : %d\n", a < b);  
    printf("a == b : %d\n", a == b);  
    printf("a != b : %d\n", a != b);  
}
```

Hint: some operators cannot contain spaces in the middle.

Example 8 - Logical Operators

Learning Note: Logical conditions often need parentheses to make evaluation order clear and correct.

```
void example8_logical_operators(void) {
    int score = 72;
    int attendance = 1;
    printf("Pass condition : %d\n", (score >= 50 && attendance == 1);
    printf("Extra chance   : %d\n", (score < 50 || attendance == 1));
    printf("Not passed     : %d\n", !(score >= 50));
}
```

Hint: use parentheses to control logical negation correctly.

Example 9 - Increment, Decrement, Assignment Operators

Learning Note: Compound operators such as `*=` are single tokens. Small spacing mistakes can make code invalid.

```
void example9_increment_decrement_assignment(void) {
    int counter = 3;
    counter++;
    printf("After ++counter : %d\n", counter);
    counter--;
    printf("After counter-- : %d\n", counter);
    counter *= 4;
    printf("After counter*=4: %d\n", counter);
}
```

Hint: compound operators are written without spaces.

Example 10 - Mini Combined Example

Learning Note: Function calls pass arguments in order, separated by commas. This final task combines function use and formatted output.

```
float rectangleArea(float width float height) {
    return width * height
}

void example10_mini_combined(void) {
    float w = 7.0f, h = 2.5f;
    float area = rectangleArea(w h,);
    printf("Width: %.1f, Height: %.1f\n", w, h);
    printf("Rectangle Area: %.2f\n", area)
}
```

Hint: function arguments in a call are separated by commas.